DATA SCIENCE INTERNSHIP

TASK 3

IPL Data Analysis and Visualization Using Python

Project Type: Exploratory Data Analysis (EDA)
Language: Python
Libraries: pandas, Matplotlib, Seaborn

Beginner-friendly project document with complete code, explanations, sample output guidance, step-by-step instructions, and report.

1. Project Objective

Analyze Indian Premier League (IPL) cricket data to discover patterns such as teams with the most wins, popular venues, top run-scoring batters, and toss decisions. Create charts and save summary tables.

2. Dataset Required

Use an IPL dataset containing two CSV files: `matches.csv` and `deliveries.csv`. The code below expects common columns such as season, team names, winner, venue, toss_decision, batter, batsman_runs, and match_id. Column names can differ between dataset versions.

3. Step-by-Step Instructions

1. Download an IPL dataset that contains matches.csv and deliveries.csv from a trusted source or use the files supplied by your instructor.

2. Open Google Colab or Jupyter Notebook. In Colab, upload both CSV files when prompted.

3. Install the required libraries if necessary: `pip install pandas matplotlib seaborn`.

4. Run the complete code. It prints summaries, generates charts, and saves CSV and PNG output files.

4. Complete Python Code

```
# Task 3: IPL Exploratory Data Analysis
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load files; upload them in Google Colab if they are not in the folder.
if not (os.path.exists("matches.csv") and os.path.exists("deliveries.csv")):
    try:
        from google.colab import files
        print("Upload matches.csv and deliveries.csv")
        files.upload()
    except ImportError:
        pass

if not (os.path.exists("matches.csv") and os.path.exists("deliveries.csv")):
    raise FileNotFoundError(
        "Please place matches.csv and deliveries.csv in the working folder."
    )

matches = pd.read_csv("matches.csv")
deliveries = pd.read_csv("deliveries.csv")

print("Matches shape:", matches.shape)
print("Deliveries shape:", deliveries.shape)
print("\\nMatch data preview:")
print(matches.head())

# Support common column-name variants
if "id" in matches.columns and "match_id" not in matches.columns:
    matches = matches.rename(columns={"id": "match_id"})
if "id" in deliveries.columns and "match_id" not in deliveries.columns:
    deliveries = deliveries.rename(columns={"id": "match_id"})
if "batter" not in deliveries.columns and "batsman" in deliveries.columns:
    deliveries = deliveries.rename(columns={"batsman": "batter"})

# 1. Team with the most match wins
if "winner" in matches.columns:
    wins = matches["winner"].dropna().value_counts().sort_values(ascending=False)
    print("\\nMost match wins by team:")
    print(wins.head(10))
    wins.to_csv("ipl_team_wins.csv", header=["Wins"])
    plt.figure(figsize=(10, 5))
    wins.head(10).sort_values().plot(kind="barh")
    plt.title("Top IPL Teams by Match Wins")
    plt.xlabel("Wins")
    plt.tight_layout()
    plt.savefig("ipl_team_wins.png", dpi=150)
    plt.show()

# 2. Most frequent venues
if "venue" in matches.columns:
    venues = matches["venue"].dropna().value_counts().head(10)
    print("\\nTop venues by number of matches:")
    print(venues)
    venues.to_csv("ipl_top_venues.csv", header=["Matches"])
    plt.figure(figsize=(10, 5))
    venues.sort_values().plot(kind="barh")
    plt.title("Top 10 IPL Venues")
    plt.xlabel("Number of matches")
    plt.tight_layout()
    plt.savefig("ipl_top_venues.png", dpi=150)
    plt.show()

# 3. Top run-scoring batters
if {"batter", "batsman_runs"}.issubset(deliveries.columns):
    batter_runs = deliveries.groupby("batter")["batsman_runs"].sum()
    batter_runs = batter_runs.sort_values(ascending=False).head(10)
    print("\\nTop batters by runs:")
    print(batter_runs)
```

```
    batter_runs.to_csv("ipl_top_batters.csv", header=["Runs"])
    plt.figure(figsize=(10, 5))
    batter_runs.sort_values().plot(kind="barh")
    plt.title("Top 10 IPL Batters by Runs")
    plt.xlabel("Runs")
    plt.tight_layout()
    plt.savefig("ipl_top_batters.png", dpi=150)
    plt.show()

# 4. Toss decision summary
if "toss_decision" in matches.columns:
    toss = matches["toss_decision"].value_counts()
    print("\\nToss decisions:")
    print(toss)
    toss.to_csv("ipl_toss_decisions.csv", header=["Count"])
    plt.figure(figsize=(6, 4))
    sns.barplot(x=toss.index, y=toss.values)
    plt.title("IPL Toss Decisions")
    plt.xlabel("Decision")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.savefig("ipl_toss_decisions.png", dpi=150)
    plt.show()

print("\\nAnalysis complete. CSV summaries and charts are saved in the working folder.")
```

5. Code Explanation

Load data: pandas reads the match-level and ball-by-ball CSV files.

Team wins: value_counts() counts how often each team appears in the winner column.

Venues: value_counts() identifies venues with the highest number of matches.

Batter runs: groupby() and sum() add the runs credited to each batter across deliveries.

Toss decisions: value_counts() summarizes how often teams chose to bat or field.

Charts and files: Matplotlib and Seaborn create bar charts, while to_csv() saves numerical summaries.

6. Sample Output Format

The actual rankings depend on the dataset version. Your program will display results in a format similar to the following:

```
Matches shape: (number_of_rows, number_of_columns)
Deliveries shape: (number_of_rows, number_of_columns)

Most match wins by team:
Team A    ...
Team B    ...

Top venues by number of matches:
Venue A   ...

Top batters by runs:
Batter A  ...

Toss decisions:
field     ...
bat       ...

Analysis complete. CSV summaries and charts are saved in the working folder.
```

The values above are placeholders, not actual calculated results. Run the code against your dataset for real totals.

7. Output Files

• CSV summaries: ipl_team_wins.csv, ipl_top_venues.csv, ipl_top_batters.csv, ipl_toss_decisions.csv (where the corresponding columns exist).

• Charts: ipl_team_wins.png, ipl_top_venues.png, ipl_top_batters.png, ipl_toss_decisions.png (where the corresponding columns exist).

8. Project Report

Title: IPL Data Analysis and Visualization Using Python

Introduction: The Indian Premier League produces match and ball-by-ball data that can be explored to understand team performance, venue usage, batting totals, and toss choices.

Methodology: Two CSV files are loaded and inspected. Match data is used to summarize winners, venues, and toss decisions. Delivery data is grouped by batter to calculate total runs. Results are displayed and visualized using bar charts.

Results: The program identifies the leading teams, venues, and batters in the chosen dataset and saves summaries and charts. The exact findings depend on the seasons and data source included.

Conclusion: This project demonstrates data loading, data inspection, aggregation, exploratory analysis, visualization, and exporting results. Dataset documentation should be checked because column names and scoring conventions may vary.

9. Troubleshooting

• FileNotFoundError: confirm both CSV files are uploaded and named exactly matches.csv and deliveries.csv.

• KeyError: your dataset may use different column names. Inspect print(matches.columns) and print(deliveries.columns), then update the code.

• ModuleNotFoundError: run pip install pandas matplotlib seaborn.

• No chart appears: check the saved PNG files in the notebook's current working directory.